

Machine-Learning Interatomic Potentials in PyTorch

Part 1 — The Theory (and why PyTorch is the natural tool)

A course built for your goal: understand MLIPs deeply enough to (a) build a new one from scratch and (b) distill a large foundation model into a small, fast one for your own chemical system. You know Python and NumPy; we assume you also know the physics side (energies, forces, DFT, MD) reasonably well. This first part is deliberately theory-heavy — almost no code — because everything we build later is just this theory expressed in PyTorch. There is one short, runnable snippet at the very end so the key idea lands in code.

0. The whole journey, so you can see where we are

We will go through six parts. You are in Part 1.

Part	Title	What you get
1	Theory (this document)	What an MLIP is as a learning problem; the symmetries it must obey; forces as gradients; the loss; a map of the architectures. No PyTorch coding yet.
2	PyTorch foundations	Tensors, autograd, <code>nn.Module</code> , a training loop — taught <i>through</i> physics examples (compute a force from a potential; fit a tiny energy model).
3	From atoms to model inputs	Neighbor lists, cutoffs, periodic boundaries, and descriptors — turning a structure into something a network can eat, without breaking the symmetries.
4	A working MLIP from scratch	A complete Behler–Parrinello-style potential in pure PyTorch: per-atom energies, forces via autograd, energy+force training on a toy dataset.
5	Modern architectures (map + one build)	Message-passing and equivariant models (SchNet → NequIP/MACE): the ideas, and how the frameworks build on Part 4.
6	Distilling a foundation model	Teacher–student setup, generating labels from a big model, energy/force (and feature/Hessian) matching, and validating the small student.

Everything in Parts 3–6 is the theory below, made concrete. If a later step ever feels like magic, the explanation is almost always in this document.

1. What an MLIP is, stated as a learning problem

An interatomic potential is a function that takes an atomic configuration and returns its **potential energy**. From that single scalar, everything else you need for simulation follows by differentiation.

The input. A configuration is:

- atomic positions $\mathbf{R} = \{\mathbf{r}_1, \dots, \mathbf{r}_N\}$, an $(N, 3)$ array of coordinates (Å),
- chemical species $\mathbf{Z} = \{Z_1, \dots, Z_N\}$, the atomic numbers,
- and, for a periodic system, the unit **cell** (a 3×3 matrix of lattice vectors).

The output. A single number, the total potential energy:

$$E = E_{\theta}(\mathbf{R}, \mathbf{Z}, \text{cell}) \quad (\text{a scalar, in eV})$$

where θ are the parameters we will learn. From E we get, *by differentiation only*:

$$\begin{aligned} \text{Forces: } \mathbf{F}_i &= - \partial E / \partial \mathbf{r}_i && (\text{an } (N, 3) \text{ array; the force on each atom}) \\ \text{Stress: } \boldsymbol{\sigma} &= (1/V) \cdot \partial E / \partial \boldsymbol{\epsilon} && (\text{a } 3 \times 3 \text{ tensor; needed for cells that can} \\ &&& \text{change shape}) \end{aligned}$$

The force is minus the gradient of the energy with respect to the atoms' positions. The stress is the gradient with respect to a strain $\boldsymbol{\epsilon}$ of the cell. **You do not learn $\mathbf{F}$ and $\boldsymbol{\sigma}$ with separate networks.** You learn *one* energy function and take its derivatives. This single fact is why PyTorch — an automatic-differentiation engine — is the natural home for MLIPs. Hold onto it; Section 3 is entirely about it.

Why learn this at all. Density Functional Theory (DFT) gives you E , $\mathbf{F}$, $\boldsymbol{\sigma}$ at roughly chemical accuracy, but at a cost that scales steeply ($\approx O(N^3)$) with system size, so it caps out at a few hundred atoms for a few picoseconds. Classical force fields are cheap and $O(N)$ but rigid and often inaccurate for anything they weren't hand-tuned for. An MLIP is the bargain in between: **fit a flexible function to DFT data once, then evaluate it at near-classical cost with near-DFT accuracy**, scaling linearly so you can reach 10^4 – 10^6 atoms and nanoseconds. That is the entire value proposition.

The dataset. Supervised learning needs labels. Here the labels come from DFT single-point calculations on many configurations:

$$\text{Dataset } D = \{ (\mathbf{R}, \mathbf{Z}, \text{cell}) \rightarrow (E_{\text{ref}}, \mathbf{F}_{\text{ref}}, \boldsymbol{\sigma}_{\text{ref}}) \}_{k=1..K}$$

Each configuration contributes **1 energy label + 3N force labels + up to 6 stress labels**. That force count is a gift: a 100-atom cell gives you 1 energy number but 300 force numbers, so *force-matching is where most of the training signal lives*. We will lean on this hard.

2. The four things your model is *not allowed* to get wrong

This is the heart of MLIP theory, and it is what separates a physics potential from a generic regression. Physical energy has exact symmetries. If your model does not build them in, it will waste data relearning them approximately, break conservation laws, and behave unphysically in MD. Each symmetry maps to a concrete architectural choice.

2.1 Translation invariance

Shift every atom by the same vector $\mathbf{t}$ and the energy must not change:

$$E(\mathbf{R} + \mathbf{t}, \mathbf{Z}) = E(\mathbf{R}, \mathbf{Z}) \quad \text{for all } \mathbf{t}$$

Absolute coordinates are therefore forbidden as inputs — a network that reads raw (x, y, z) would see a different input for the same physical state. **Enforcement:** feed the model only *relative* quantities — displacement vectors $\mathbf{r}_{ij} = \mathbf{r}_j - \mathbf{r}_i$, or the distances $r_{ij} = |\mathbf{r}_{ij}|$. These are unchanged by a global shift.

2.2 Rotational (and reflection) symmetry — $O(3)$

Rotate the whole system by any rotation matrix Q . The **energy is a scalar**, so it must be invariant:

$$E(\mathbf{R} Q^T, \mathbf{Z}) = E(\mathbf{R}, \mathbf{Z})$$

but the **forces are vectors**, so they must be *equivariant* — they rotate along with the system:

$$\mathbf{F}(\mathbf{R} Q^T, \mathbf{Z}) = \mathbf{F}(\mathbf{R}, \mathbf{Z}) Q^T$$

This invariant-vs-equivariant distinction is the single most important idea in modern MLIP architecture, so be precise about it:

- **Invariant** quantity: unchanged when you rotate the input (a distance, an angle, the energy).
- **Equivariant** quantity: transforms *the same way* the input did (a force, a velocity, a dipole).

There are two ways to respect this:

1. **Use only invariant features.** Distances and angles are rotation-invariant. Build the energy purely from them and it is automatically invariant; the forces then come out correctly equivariant *because they are derivatives of an invariant scalar* (differentiation preserves this). This is the first-generation approach (symmetry functions, SOAP) and the one we build in Part 4.
2. **Use equivariant features and only ever contract them into scalars for the energy.** Keep directional information (vectors, higher tensors) flowing through the network, transform it correctly at every layer, and combine it into invariants right at the energy readout. This is the modern equivariant-GNN approach (NequIP, MACE), and it is more data-efficient because directional information is preserved instead of thrown away. It needs the machinery of spherical harmonics and tensor products (the [e3nn](#) library); we cover the ideas in Part 5.

2.3 Permutation invariance

Two identical atoms are interchangeable. Swapping the labels of two carbon atoms cannot change the energy:

$$E(\dots, \mathbf{r}_i, \dots, \mathbf{r}_j, \dots) = E(\dots, \mathbf{r}_j, \dots, \mathbf{r}_i, \dots) \quad \text{for atoms of the same species}$$

Enforcement: never let the answer depend on atom ordering. Two devices give this for free:

- **Per-atom energy decomposition** (below): the total is a *sum* over atoms, and addition doesn't care about order.
- **Symmetric neighbor aggregation:** whenever an atom looks at its neighbors, it combines them with an order-independent operation (sum or mean), never anything that depends on the list order.

2.4 Locality, smoothness, and extensivity (the assumptions that make it scale)

Three more properties, slightly different in kind — they are modeling assumptions rather than exact symmetries, but they are what make an MLIP fast and transferable.

Locality. We assume an atom's contribution to the energy depends only on its neighbors within a **cutoff radius** r_c (typically 4–6 Å). Formalized as a **per-atom energy decomposition**:

$$E = \sum_i E_i, \quad \text{where } E_i = E_i(\text{local environment of atom } i \text{ within } r_c)$$

Each E_i is computed from only the atoms near atom i . This is the workhorse equation of the entire field. It buys you three things at once: **permutation invariance** (a sum), **extensivity / size-consistency** (double the system, double the energy — because you just sum more terms), and **linear scaling** (each atom does a bounded amount of work, so cost $\propto N$). It also gives **transferability**: a model trained on small cells can be applied to large ones, because it only ever reasons about local environments.

Smoothness. Forces are derivatives of the energy, so if the energy has a kink, the force has a jump and MD blows up. In particular, when a neighbor crosses the cutoff r_c , its contribution must fade to zero *smoothly*, not switch off abruptly. **Enforcement:** multiply every neighbor's contribution by a smooth **cutoff function** $f_c(r)$ that goes to 0 (with zero slope) at r_c — a cosine window or a polynomial envelope. Small detail, non-negotiable in practice.

Here is the mental picture for the whole section:

```
translation → use relative vectors  $r_{ij}$ 
rotation    → invariant features (distances/angles) OR equivariant
features → scalar readout
permutation →  $E = \sum_i E_i$  and sum over neighbors
locality    → each  $E_i$  sees only neighbors within  $r_c$ 
smoothness  → multiply neighbor terms by a smooth cutoff  $f_c(r)$ 
```

If you remember only one diagram from Part 1, remember that one.

3. The central trick: forces are gradients, and autograd computes them *exactly*

This is where PyTorch stops being "a way to code the model" and becomes "the reason the whole approach works."

3.1 The math

Energy is a scalar function of the 3N position coordinates. The force on atom i is minus the gradient of that scalar with respect to atom i 's coordinates:

$$\mathbf{F}_i = - \partial E / \partial \mathbf{r}_i \quad (\text{three components: } -\partial E / \partial x_i, -\partial E / \partial y_i, -\partial E / \partial z_i)$$

Two consequences that people new to MLIPs often miss:

- **The forces are not a separate prediction.** They are a mathematical property of the energy surface you already learned. If your energy is a smooth, differentiable function of positions, the forces exist and are fully determined.
- **This guarantees energy conservation.** Because $\mathbf{F}$ is *exactly* the negative gradient of a single scalar E , the force field is *conservative* by construction — energy is conserved in an NVE molecular-dynamics run, momentum is conserved, and there is a well-defined potential-energy surface. (Some newer models predict forces directly with a separate head for speed; they gain a little throughput but *lose* this guarantee. Building forces as gradients, as we will, is the physically safe default.)

3.2 Why PyTorch

Automatic differentiation ("autograd") is a technique for computing exact derivatives of any function you can express as code, by recording the elementary operations and applying the chain rule mechanically. It is **not** finite differences (no $(E(\mathbf{r}+\mathbf{h}) - E(\mathbf{r}))/h$, no step-size error, no 3N extra energy evaluations) and it is **not** symbolic math (no giant formula). It is exact, to machine precision, and costs about the same as evaluating E once.

So the recipe for an MLIP is:

1. Write the energy $E = E_\theta(\mathbf{R})$ as a differentiable PyTorch function (a neural network).
2. Ask autograd for $\partial E / \partial \mathbf{R}$.
3. Negate it. Those are your forces — exact, conservative, essentially free.

That is *literally* the whole force computation. In Part 4 it is one line. The preview in Section 7 shows it working on a potential whose forces you can check by hand.

3.3 The one subtlety that trips everyone up (second derivatives)

Here is the wrinkle, and it is worth understanding now because it explains a flag you will see everywhere in MLIP code.

We don't just *evaluate* forces — we *train on them*. The loss contains a force term, $\|\mathbf{F}_{\text{pred}} - \mathbf{F}_{\text{ref}}\|^2$. But $\mathbf{F}_{\text{pred}} = -\partial E / \partial \mathbf{R}$ is *itself a first derivative* of the network. To update the weights θ we need $\partial \text{Loss} / \partial \theta$, which means differentiating the loss — and therefore differentiating $\mathbf{F}_{\text{pred}}$ — with respect to θ . That is a **derivative of a derivative**: a second-order quantity.

Concretely, one training step needs derivatives in two different directions:

first derivative w.r.t. POSITIONS	→	gives the forces	(the physics)
then derivative w.r.t. PARAMETERS	→	updates the weights	(the learning)

For the second pass to be possible, the force computation must remain part of the differentiable graph. In PyTorch that is exactly what the flag `create_graph=True` does when you call `torch.autograd.grad(E, positions, create_graph=True)`. It says: "don't throw away the computation you just did to get the forces — I'm going to differentiate through it again." Forget that flag and force-training silently fails (you'll get zero or an error at `loss.backward()`); remember it and everything works. We'll meet it properly in Part 4 — for now, just know *why* it exists: forces are first derivatives, and training on them needs a second.

4. The training objective

We fit θ by minimizing a weighted sum of energy, force, and (for periodic systems) stress errors over the dataset:

$$L(\theta) = w_E \cdot L_E + w_F \cdot L_F + w_\sigma \cdot L_\sigma$$

with, typically, mean-squared errors:

$L_E = \text{mean over configs of } ((E_{\text{pred}} - E_{\text{ref}}) / N_{\text{atoms}})^2$	← per-atom, so big and small cells compare fairly
$L_F = \text{mean over all atoms \& xyz of } \ F_{\text{pred}} - F_{\text{ref}}\ ^2$	← 3N components per config: the bulk of the signal
$L_\sigma = \text{mean over the 6 independent stress components of } (\sigma_{\text{pred}} - \sigma_{\text{ref}})^2$	

Four practical points that matter more than they look:

- **Weighting.** The relative sizes of w_E , w_F , w_σ are a real modeling choice. Forces are usually weighted heavily because they carry local information and are what MD actually integrates; a common pattern even *anneals* the weights during training (force-heavy early, energy-heavy late).
- **Per-atom energy.** Dividing the energy error by N_{atoms} keeps a 1000-atom cell from dominating a 10-atom molecule purely by size. Forces are already intensive (per atom), so they're just averaged.
- **Reference energy shifts.** Raw DFT total energies are enormous and dominated by uninteresting core/atomic contributions. Before training you subtract a per-element baseline $E_0(Z)$ (isolated-atom energies, or values fit by linear regression) so the model learns the *cohesive/formation* energy — the interesting part. This dramatically improves conditioning and is standard. You add the baseline back at prediction time.
- **Normalization.** Forces are often divided by their dataset RMS so the loss is well-scaled. Small bookkeeping, big difference in training stability.

Everything above is architecture-agnostic: it is the objective whether your energy model is a 1-line linear map or a 50-million-parameter equivariant GNN. Only E_{pred} changes between models. Which brings us to the models.

5. A map of the architectures (so the whole field has a shape)

Every MLIP computes per-atom energies E_i from local environments and sums them (Section 2.4). They differ in **how E_i is computed from the neighborhood** — specifically, in how they turn raw geometry into rotation-invariant information without discarding too much. There are two families and a "third generation" you'll be distilling.

5.1 First generation — fixed descriptors + a small network

Hand-design a set of **invariant descriptors** of each atom's environment, then map them to an energy with a small multilayer perceptron (MLP), one per element.

```
neighborhood of atom i —(fixed, hand-designed transform)—► descriptor vector
—MLP—► E_i
```

- **Behler–Parrinello Neural Network (BPNN, 2007)** — the original. The descriptors are **Atom-Centered Symmetry Functions (ACSFs)**: radial functions G^2 (sums over neighbor distances) and angular functions G^4/G^5 (sums over neighbor-pair angles), all invariant by construction. Simple, robust, still widely used. **This is what we build in Part 4**, because every idea in it is transparent — you can see exactly where each symmetry is enforced.
- **SOAP + GAP** — a richer descriptor (Smooth Overlap of Atomic Positions) with a Gaussian-process regressor instead of a neural net. Same philosophy: fixed invariant features, learned mapping.

The signature limitation: the features are *fixed*, chosen by you, not learned from data. Expressive power is capped by how good your hand-designed descriptors are.

5.2 Second/third generation — message-passing graph neural networks

Represent the structure as a **graph**: atoms are nodes, and an edge connects two atoms if they are within the cutoff. Then *learn* the features by passing "messages" along edges and updating each atom's state over several rounds. The network discovers its own descriptors.

```
build graph (edges within r_c) —► repeat T times: each atom aggregates
messages from neighbors, updates its state
                                —► read out E_i from each atom's final state
—► E = Σ_i E_i
```

Within this family, the axis that matters is *what kind of information the messages carry*:

- **Invariant GNNs — e.g. SchNet.** Messages are built only from *distances* (via smooth radial basis functions and "continuous-filter" convolutions). Everything stays rotation-invariant the easy way. Good, simple, a natural first GNN to understand.
- **Equivariant GNNs — PaiNN, NequIP, Allegro, MACE.** Messages carry *directional* information (vectors, and higher tensors), transformed correctly at every layer using **spherical harmonics** and **tensor products** (the `e3nn` library encodes the $O(3)$ representation theory so you don't have to). Because they

keep directional detail instead of collapsing to distances immediately, they are strikingly **data-efficient** — often reaching high accuracy with far fewer training configs. **MACE** additionally packs *higher body-order* interactions into each layer (via the Atomic Cluster Expansion), so a couple of layers capture what would otherwise need many — this is why MACE is both accurate and comparatively fast, and why it underpins several foundation models.

You do **not** need to implement equivariance from scratch to use it — that is what **e3nn** and the frameworks are for. Part 5 gives you the concepts and shows how they sit on top of the same $E = \sum E_i$, forces-by-autograd skeleton you'll already own from Part 4.

5.3 Third generation in practice — *foundation* (universal) MLIPs

Recently the field produced **universal potentials**: single models trained on enormous DFT databases spanning much of the periodic table, intended to work "out of the box" on almost any inorganic (or organic) system. Representative examples you'll encounter:

- **MACE-MP** (equivariant MACE trained on the Materials Project trajectory set, MPTrj),
- **CHGNet** (graph net that also predicts magnetic moments; MPTrj),
- **M3GNet**, **MatterSim**, **SevenNet**, **ORB**, and **graph-ACE / GRACE** models, among others.

These are powerful but relatively heavy and slow for large-scale or long-time MD, and being universal, they are not specialized to *your* system. That is the exact gap **distillation** fills — and your stated goal.

The universal-model landscape moves fast, and specific model names, versions, and licenses change month to month. When we reach Part 6 I'll pull current, accurate details (which models are available, their APIs, and the up-to-date distillation tooling) rather than rely on any fixed list — recent 2026 work explicitly on distilling these into lightweight potentials confirms this is a live, validated technique, not a hypothetical.

6. Where we're heading — the two things you actually want to do

6.1 Build a new MLIP from scratch (Parts 2–4)

The plan, all of which is just the theory above turned into PyTorch:

1. Turn structures into inputs without breaking symmetry: neighbor lists within r_c , relative vectors, smooth cutoff (Part 3).
2. Compute invariant descriptors of each atom's environment (Behler–Parrinello style) (Part 3–4).
3. Map descriptors $\rightarrow$ per-atom energy E_i with a small per-element MLP; sum to E (Part 4).
4. Get forces as $-\partial E / \partial R$ via autograd, with `create_graph=True` for training (Part 4).
5. Train on the combined energy+force loss; validate on held-out configs and on physics (Part 4).

6.2 Distill a foundation model into a small, fast one (Part 6)

The plan, which reuses the *entire* build above as the "student":

1. Pick a **teacher**: a pretrained foundation MLIP (frozen — we never train it).
2. Pick/design a **student**: a small, fast architecture (could be your Part-4 model, or a shallow MACE) specialized to *your* chemical system.

3. **Generate configurations** that cover where you'll actually use the model — run MD with the teacher, rattle structures, apply strains. This is cheap because *no new DFT is needed*: the teacher provides the labels.
4. **Label** those configs with the teacher's E, F (and σ).
5. **Train the student to match the teacher** using the same energy+force loss — optionally augmented with *feature matching* (make the student's internal representations imitate the teacher's) and *Hessian/curvature matching* (match second derivatives for better phonons), and optionally a little real DFT to correct teacher bias.
6. **Validate**: energy/force MAE of student vs. teacher, physical tests (phonons, elastic constants, MD stability), and — the whole point — measure the **speed-up** (ms/step, atoms/second).

Knowledge distillation works here for the same reason it works elsewhere: the teacher's labels are *smooth and dense*. Instead of a handful of expensive DFT points, the student learns from an unlimited supply of cheap, self-consistent teacher labels exactly in the region of configuration space you care about — so a much smaller network can match the teacher's accuracy *on your system* while running many times faster.

7. One snippet, so the key idea is concrete

You asked for theory first, then a slow introduction to code — so here is the *single* most important line of the whole course, in context, with everything else stripped away. We use a Lennard-Jones pair energy (a known formula with a force you can verify by hand) *in place of* a neural network, purely so you can see that autograd returns the right force. In Part 4, `energy(...)` becomes a network and **nothing else about this pattern changes**.

```
import torch

# Two atoms, 3D coordinates in Ångström. requires_grad=True tells PyTorch:
# "track operations on this tensor so I can differentiate w.r.t. it later."
positions = torch.tensor([[0.0, 0.0, 0.0],
                          [1.2, 0.0, 0.0]], requires_grad=True)  # shape (N=2, 3)

def energy(pos):
    r = torch.linalg.norm(pos[0] - pos[1])  # interatomic distance
    (translation-invariant!)
    return 4.0 * (r**-12 - r**-6)  # Lennard-Jones energy
    (eps=sigma=1). A NN goes here later.

E = energy(positions)  # forward pass: one scalar energy

# THE LINE. F = -dE/dR, computed exactly by autograd. This is the entire force
# calculation.
forces = -torch.autograd.grad(E, positions)[0]  # shape (2, 3)

print("Energy:", E.item())
print("Forces:\n", forces)
```

Note what's already visible here, straight from the theory:

- The energy is built from a **distance**, so it's automatically **translation- and rotation-invariant** (Section 2.1–2.2).
- The force is **$-\text{grad}(E)$** — one call, exact, no finite differences (Section 3).
- By Newton's third law the two forces must be **equal and opposite** and lie along the bond (the x-axis). When you run this, they will be — which is your first proof that "forces = negative gradient of energy" is not just tidy notation but a thing autograd delivers correctly.

Verified output (I ran exactly this code in PyTorch 2.13 before handing it to you):

```
Energy: -0.890965
Forces:
[[ 2.2116942  0.  0. ]
 [-2.2116942  0.  0. ]]
```

Read what that confirms: the two force vectors are **equal and opposite** (Newton's third law, $F_0 = -F_1$), they point **purely along x** (the bond direction — the y and z components are exactly 0), and the magnitude 2.211694 matches the hand-derived Lennard-Jones derivative $|dE/dr|$ at $r = 1.2$ to six significant figures. Atom 0 is pushed toward atom 1 ($r = 1.2$ sits just past the LJ minimum at ≈ 1.122 , so the pair is in its attractive region). You just watched autograd produce a physically exact force from an energy, with one function call and no calculus on your part. Every MLIP in this course rests on that.

8. Notation & vocabulary cheat-sheet

Symbol / term	Meaning
$\mathbf{R}, \mathbf{r}_i$	all positions ($N \times 3$); position of atom i
$\mathbf{Z}, Z_i$	species (atomic numbers)
E, E_i	total energy; per-atom energy contribution ($E = \sum_i E_i$)
$\mathbf{F}_i$	force on atom i , $= -\partial E / \partial \mathbf{r}_i$
σ	stress tensor, from $\partial E / \partial (\text{strain})$
r_c	cutoff radius (locality assumption)
$\mathbf{r}_{ij}, \mathbf{r}_{ij}$	distance / displacement vector between atoms i and j
$f_c(r)$	smooth cutoff function ($\rightarrow 0$ at r_c)
invariant	unchanged under a symmetry operation (distance, angle, energy)
equivariant	transforms <i>with</i> the operation (force, dipole)
θ	the model's learnable parameters
autograd	automatic differentiation; exact derivatives of code
<code>create_graph=True</code>	keeps the force computation differentiable so you can <i>train</i> on forces
descriptor	a fixed, invariant summary of an atom's environment (e.g. ACSF, SOAP)

Symbol / term	Meaning
message passing	learning environment features by exchanging info along graph edges
ACSF / BPNN	Behler–Parrinello symmetry functions / neural network (first-gen MLIP)
SchNet / NequIP / MACE	invariant / equivariant graph MLIPs
foundation (universal) MLIP	a big pretrained potential covering much of the periodic table (MACE-MP, CHGNet, ...)
teacher / student	the big pretrained model / the small model we train to imitate it (distillation)

9. Check yourself before Part 2

If you can answer these in a sentence each, you're ready for code. If not, the relevant section is in parentheses.

1. Why can't atomic positions be fed to the network directly? (2.1)
2. What is the difference between an *invariant* and an *equivariant* quantity, and which is the energy, which is the force? (2.2)
3. What does the decomposition $E = \sum_i E_i$ buy you? (Name all three benefits.) (2.4)
4. Why must a neighbor's contribution fade smoothly at the cutoff? (2.4)
5. How are forces obtained from the model, and why does that guarantee energy conservation? (3.1)
6. Why does *training* on forces require second derivatives, and what flag enables that in PyTorch? (3.3)
7. Why is the energy error usually divided by the number of atoms in the loss? (4)
8. In one line: what is model distillation doing for you here, and why does it let a *small* model match a big one on your system? (6.2)

Next: **Part 2 — PyTorch foundations, taught through physics.**

Tensors as "NumPy arrays that remember how they were made," autograd from the ground up, `nn.Module`, and a first end-to-end training loop that fits a tiny energy model — so that by the end you can read every line of the from-scratch MLIP in Part 4. Tell me when you want it and I'll build it the same way: concept first, then the code.